Smart Waste Management System For Metropolitan Cities

Final Year Project Documentation

SmartInternz Externship Program 2026

Submitted By: Avishkar Ashok Renuse.

Project Domain: IoT, Smart Cities, Web Development, AI

1. Project Overview

The **Smart Waste Management System (SWMS)** is an advanced full-stack web application that simulates a real-world IoT-powered waste monitoring platform for metropolitan cities. It integrates real-time bin fill-level tracking, AI-based route optimization, live analytics dashboards, and a REST API for IoT device connectivity.

The system monitors 12 smart bins distributed across 6 zones of the **Pune Metropolitan Region**, providing municipal authorities with actionable insights to optimize collection schedules, reduce fuel costs, and maintain city hygiene standards. Built using Python Flask, Leaflet.js maps, Chart.js analytics, and a professional dark-sidebar UI, this project demonstrates a complete end-to-end smart city solution aligned with UN Sustainable Development Goals 11, 12, 14, and 15.

2. Objectives

- **Design and develop** a real-time IoT bin monitoring system using simulated ultrasonic sensor data.
- **Implement** an AI-powered route optimization algorithm using Greedy NearestNeighbour logic with Haversine distance calculations.
- **Build** a professional analytics dashboard featuring live charts, KPI cards, and zonewise analysis.
- **Create** an interactive city map with color-coded bin markers and animated truck route visualizations.
- **Develop** a complete REST API for IoT device integration and mobile app connectivity.
- **Align** the core solution with UN SDG 11 (Sustainable Cities) and SDG 12 (Responsible Consumption).

3. Technologies Used

Layer	**Technology**	**Purpose**
Backend	Python 3 + Flas	Web server, routing, and REST API development
Frontend	HTML5 + Cust CSS3	Professional dark-sidebar user interface
Maps	Leaflet.js + OpenStreetMap	Interactive city map tracking bin markers
Charts	Chart.js 4.x	Dynamic bar, doughnut, pie, and line charts
Algorithm	Greedy NearestNeighbo	Optimal collection truck route planning
Distance	Haversine Form	GPS-accurate distance calculations
IoT Sim	Time-seeded RI	Realistic fill-level behavior simulation
API	Flask REST (JS	IoT device and mobile application integration
Version Control	Git & GitHub	Source code management

4. Folder Structure

Plaintext

```
Smart-Waste-Management-System/
│
├── Documentation/
│   └── Project Documentation.docx
│
├── static/
│   └── css/
│       └── style.css
```

```
│
├── templates/
│   ├── sidebar.html        # Reusable sidebar component
│   ├── home.html           # Landing page
│   ├── dashboard.html      # Live analytics dashboard
│   ├── map.html            # Interactive map route
│   ├── bins.html           # Bin monitoring grid
│   └── reports.html        # Weekly KPI reports
│
├── app.py                  # Main Flask application
├── requirements.txt └──
README.md
```

(Structure adapted from)

5. System Architecture

5.1 Overall Flow

1. IoT sensors (simulated) report bin fill levels every 15 minutes.
2. The Flask backend processes the incoming data, computes the optimal route, and generates alerts.
3. Jinja2 templates render the live data cleanly to the web browser.
4. Client-side JavaScript (Chart.js, Leaflet.js) renders the interactive visual assets.
5. REST API endpoints allow external IoT devices and mobile apps to push or pull data.

5.2 Bin Network Deployment

The system actively coordinates 12 Smart Bins across 6 designated zones:

Bin II	Location	Zone	Waste T	Capacity
BIN0	MG Road Jun	Zone	General	100 L
BIN0	FC Road Marl	Zone	Recycla	100 L
BIN0	Shivaji Nagar	Zone	Organic	150 L
BIN0	Koregaon Parl	Zone	General	100 L
BIN0	Baner Tech H	Zone	Recycla	200 L
BIN0	Hinjewadi IT	Zone	General	200 L
BIN0	Wakad Marke	Zone	Organic	100 L
BIN0	Kothrud Squa	Zone	General	150 L
BIN0	Sinhagad Roa	Zone	Recycla	100 L
BIN0	Hadapsar Indu	Zone	Hazardo	200 L
BIN0	Viman Nagar	Zone	General	100 L
BIN0	Aundh Centra	Zone	Organic	150 L

6. Key Features

6.1 Live Dashboard

- **4 KPI Stat Cards:** At-a-glance metrics for Total Bins, Critical Bins ($\ge85\%$), Warning Bins (60-84%), and Average City Fill Percentage.
- **Chart.js Grouped Bar Chart:** Compares weekly waste collected vs. recycled (in kg).
- **Doughnut Chart:** Visualizes current bin status distribution (OK / Warning / Critical).
- **Real-time Alert Feed:** Shows urgent updates with severity icons and timestamps.
- **Zone-wise Grid:** Tracks the average fill percentage across all 6 city zones.
- **Actionable Critical Table:** Lists high-priority bins with an immediate collection action button.

6.2 Interactive Map

- Integrated Leaflet.js map centered precisely on the **Pune Metropolitan Region**.
- **Color-Coded Status Markers:** Red (Critical), Yellow (Warning), and Green (OK).
- **Dynamic Popup Cards:** Clickable components revealing fill levels, waste type, and predicted time until full.
- **AI Route Polyline:** Displays an animated dashed polyline tracking the optimized truck route.
- **Numbered Stops:** Visually labels route milestones sequential order ($1, 2, 3\dots$).
- **Route Statistics:** Live tracker for total stop counts, total distance (km), and estimated travel time.

6.3 Bin Monitor Grid □ A fully responsive card layout showing all 12 individual bins equipped with precise fill bars and status badges.

- Advanced filtering systems categorized by **Zone (A-F)** and **Waste Type** (General, Recyclable, Organic, Hazardous).
- Heuristic-driven predictive countdown estimating time remaining until bins reach capacity.
- Color-coded top borders corresponding directly to live bin status severity.

6.4 Reports & KPIs

- **Analytical Visuals:** Bar charts mapping weekly collection trends and pie charts detailing waste type distributions.
- **Historical Tracking:** A 7-day line chart mapping city-wide fill levels.
- **Aggregated Metrics:** Total waste collected, total volume recycled, overall recycling rate percentage, and active bin numbers.

7. Route Optimization Algorithm

7.1 Greedy Nearest-Neighbour

The core software leverages a **Greedy Nearest-Neighbour** routing algorithm to plot the collection vehicle’s path. To optimize municipal fuel resources and minimize traffic footprint, **only bins with Critical status ($\ge85\%$ full) are injected into the active pathing array**.

7.2 Haversine Distance Formula

To compute distance matrices across live GPS coordinates, the system utilizes the mathematical **Haversine formula**, accounting elegantly for the Earth's spherical curvature:

$$d = 2R \arcsin \left(\sqrt{\sin^2\left(\frac{\Delta \text{lat}}{2}\right) + \cos(\text{lat}_1) \cos(\text{lat}_2) \sin^2\left(\frac{\Delta \text{lng}}{2}\right)}\right)$$

The computed distance output maps accurately in kilometers, which allows the application to project highly realistic travel times based on a standard fleet vehicle averaging a city speed of 30 km/h.

8. IoT Sensor Simulation

Each physical bin unit relies on a specialized time-seeded random number generator designed to replicate genuine ultrasonic sensor behaviors:

- **Hourly Consistency:** Generated levels remain uniform within any specific hour block, mirroring real hardware capturing samples every 15 minutes.
- **Accumulation Trends:** Generated values scale smoothly across sequential hour variations to map true-to-life urban garbage accumulation.
- **Unique Seeding:** Every device maintains a unique seed tied explicitly to its unique `Bin ID`, ensuring unique data feeds across different locations.
- **Threshold Triggers:**
  - **OK:** $<60\%$
  - **Warning:** $60\% - 84\%$
  - **Critical:** $\ge85\%$

9. REST API Reference

Endpoint	Method	Description	Response Format

`/api/bins`	`GET`	Fetches all statuses and fil	JSON Array
`/api/bins/<`	`GET`	Fetches real-ti data for a singl	JSON Object
`/api/bins/<`	`POST`	Entry point for IoT hardware t	Action Decision
`/api/route`	`GET`	Pulls the collection path	Ordered JSON List
`/api/stats`	`GET`	Provides aggregated dat	Summary JSON

9.1 Sample API Response (`/api/stats`)

JSON
```
{
  "total_bins": 12,
  "critical": 4,
  "warning": 3,
  "ok": 5,
  "avg_fill": 61.5 }
```

(Adapted from)

10. Setup & Run Instructions

To deploy and execute the platform locally, follow these steps:

Step 1: Install Dependencies

Ensure you have Python installed, then run the package manager via terminal:

Bash
```
pip install -r
requirements.txt
```

(Adapted from)

Step 2: Launch the Flask Application

Run the primary execution script:

Bash `python app.py`

(Adapted from)

Step 3: Access the Interface

Open your web browser and navigate to the following local address:

URL: http://localhost:5000

11. UN SDG Alignment

SDG Goal	Title	Core Project Contribution
SDG 11	Sustainable Ci	Real-time smart bin analytics prevent overflow, keeping urban pathways cleaner.
SDG 12	Responsible Consumption	Integrated recycling rate metrics foster better waste classification and tracking.
SDG 14	Life Below W	Proactive collection mitigates loose debris runoff from entering local waterways.
SDG 15	Life on Land	Optimized management drastically reduces plastic scatter and general land pollution.

12. Conclusion

The **Smart Waste Management System** successfully highlights an industry-standard implementation of modern web technology paired with intelligent routing algorithms to tackle modern urban planning crises. By fusing real-time monitoring maps, an automated route system, professional metric graphs, and accessible REST endpoints, it functions as a highly valuable tool for municipal city developers.

The codebase is engineered to scale easily. Future iterations can cleanly substitute the timeseeded simulators for live physical microcontrollers (e.g., Arduino or Raspberry Pi utilizing MQTT architecture), deploy immediate email/SMS notifications, or build Machine Learning models for predictive waste forecasting.

Project Links

- **GitHub Repository:** https://github.com/Avirenuse/Smart-Waste-Management-System-For-Metropolitan-Cities.git
- **Video Demonstration:** https://drive.google.com/file/d/1PNIbo5qOipkkm_8wXx7k6dz7HUWpKTxQ/view?usp=drive_link